Module 9 Week B — Reading: Entity Linking on Property Graphs, Text2Cypher & GNN Literacy

> ***Module 8 → Module 9.*** *Module 8 taught you how to find relevant context: a vector store retrieves passages whose embeddings are nearby in semantic space. Module 9 teaches you how to find facts and the paths that connect them: a knowledge graph returns rows you can trace back to a specific node, a specific relationship, and a specific path through the schema. Vectors are how you find candidates fast; graphs let you trace the path that connects them.*

What success looks like: By the end of Week B, you can read and write Cypher for single-node, single-hop, and variable-length-path patterns over a Neo4j property graph, implement Identity Discipline with a uniqueness constraint and `MERGE`, build a two-step entity linker with explicit NIL handling, compute precision/recall/F1 with the triple-stated methodology, describe Text2Cypher as schema-bounded NL→code generation, and recognize GAT and R-GCN message-passing equations at a conceptual level.

In This Reading

1. Setup — install before the concept session
2. Property graphs and RDF — the same graph primitive in two stacks
3. Cypher as pattern matching
4. Identity Discipline — the property-graph deliverable
5. Entity linking on a property graph
6. The NIL case and linker metrics
7. Evaluation against gold
8. Text2Cypher — schema-bounded NL→code generation
9. Text2Cypher vs full GraphRAG
10. Text2Cypher safety (Tier 3 only)
11. KG completion as a learning problem
12. GNN literacy — GAT and R-GCN
13. Neural–symbolic integration
14. The Week B Python toolkit

- Troubleshooting
- Key Terms
- Summary
- What's Next

---

1. Setup — install before the concept session

Run these checks before Sunday's concept session. Week B introduces a new piece of infrastructure — a Neo4j database running locally via Docker — plus a small Python driver. The bulk of the install time is the Neo4j Docker image; start it pulling before you read the rest of this section if your connection is slow.

> ***Hardware-profile sidebar.*** *The core Week B path runs comfortably on any laptop with* ***≥8 GB RAM****. Neo4j's memory is capped at `heap_max=1G` + `pagecache=512M`, which is what makes that work. Challenge Tier 3 (live-LLM Text2Cypher via Ollama) is much heavier — it needs* ***≥16 GB RAM*** *to run a Phi-3 Mini or Qwen 2.5 1.5B model alongside Neo4j and your editor without swap. If your laptop is on the tight side, skip Tier 3 entirely — it has no rubric impact on the core lab and integration.*

1.1 Core install (everyone)

> ***Windows users.*** *The commands below target Bash/Zsh (macOS/Linux/WSL2/Git Bash). On PowerShell or CMD:*
>
> - *Activate your venv with `.venv\Scripts\Activate.ps1` (PowerShell) or `.venv\Scripts\activate.bat` (CMD). Git Bash works with `source .venv/Scripts/activate`.*
> - *For multi-line commands ending in `\` (e.g., the `docker run …` block in 1.1.3), either join them onto one line or replace `\` with a backtick `` ` `` in PowerShell.*
> - *The `python - <<'PY' … PY` heredoc in 1.1.3 is Bash-only. PowerShell users: save the Python between `<<'PY'` and `PY` to a temporary `.py` file and run `python smoke.py` instead.*
> - *`docker pull` / `docker compose …` require Docker Desktop with the* ***WSL2 backend*** *active — confirmed in Pre-Work setup. If `docker` is not found on a fresh PowerShell session, restart your shell after Docker Desktop installs.*

Full cross-platform command tables live in the Pre-Work install plan.

1.1.1 Pull the Neo4j Docker image.

```
docker pull neo4j:5-community
```

About **600 MB**; typically **~3 minutes** on broadband, cached after the first pull. This is the same Docker you've used since the Pre-Work; no new system-level install. We use the Community edition because it ships with the Cypher engine and (from 5.11 onward) a native vector index, which is everything Week B needs. APOC and Enterprise features are not used.

1.1.2 Install the Python driver.

From your course virtualenv:

```
pip install -r requirements.txt
```

For Module 9 Week B, `requirements.txt` carries:

```
neo4j>=5.17,<6
spacy>=3.7,<4
python-dotenv>=1.0,<2
pytest>=8,<9
```

`neo4j` is the official Bolt driver — the wire protocol used to talk to a Neo4j database from Python. `spacy` is carried forward from Module 6; if you already have `en_core_web_sm` from M6 you do not need to re-download it. If you don't:

```
python -m spacy download en_core_web_sm
```

1.1.3 Bolt smoke test.

Start a local Neo4j container and verify the driver can reach it:

```
docker run --rm -d --name neo4j-smoke \
  -p 7687:7687 -p 7474:7474 \
  -e NEO4J_AUTH=neo4j/testtest \
  -e NEO4J_dbms_memory_heap_max__size=1G \
  -e NEO4J_dbms_memory_pagecache_size=512M \
  neo4j:5-community
```

Wait ~20 seconds for the Java VM to warm up, then:

```
python - <<'PY'
from neo4j import GraphDatabase
d = GraphDatabase.driver("bolt://localhost:7687", auth=("neo4j", "testtest"))
with d.session() as s:
    print(s.run("RETURN 1 AS ok").single()["ok"])
d.close()
PY
```

You should see `1`. If you see `Bolt connection refused`, the container hasn't finished warming up — wait a few more seconds and re-run. If it still fails, see Troubleshooting at the bottom of this reading.

Stop the smoke test container with `docker stop neo4j-smoke` when you're done. The Lab and Integration use a `docker-compose.yml` to manage Neo4j for you.

This same install carries through into the Lab and Integration repos via pip's cache — downstream installs are fast.

1.2 Tier 3 install (opt-in)

If you plan to attempt the Integration's Challenge Tier 3 — live-LLM Text2Cypher via Ollama — install the runtime now so you don't compress your Thursday window.

1.2.1 Install the Ollama runtime. Download the binary from the official Ollama site for your platform (macOS, Linux, Windows). The install is ~3 minutes on macOS / Linux; Windows uses the documented WSL2 path.

1.2.2 Pull a small instruct model. The Integration's Tier 3 reference answer key was tuned against one of:

```
ollama pull phi3:mini-4k-instruct-q4_K_M    # ~2.4 GB
# or
ollama pull qwen2.5:1.5b-instruct-q4_K_M    # ~1.5 GB
```

Either is fine; the Integration guide names which one the reference answer key was pinned against. First pull is **~3–10 minutes** per laptop depending on connection.

1.2.3 Install Tier 3 Python deps. From your virtualenv:

```
pip install -r requirements-challenge.txt
```

Which carries:

```
langchain>=0.2,<0.3
langchain-neo4j>=0.1,<0.2
langchain-openai>=0.1,<0.3
langchain-anthropic>=0.1,<0.3
ollama>=0.3,<0.4
```

You'll use these only inside the Integration's `challenge/` subdirectory. The core Lab and Integration deliverables are LLM-free.

---

2. Property graphs and RDF — the same graph primitive in two stacks

Week A taught you to think about knowledge as a graph of relationships. The vehicle was RDF: triples `(subject, predicate, object)` where every node and predicate is a URI, queried with SPARQL, executed by a triple store (Fuseki). Week B is the same underlying primitive — graph as pattern matching — in a second production stack: a **property graph**, queried with **Cypher**, executed by **Neo4j**.

Both stacks ship in real production systems. Neither replaces the other:

- **RDF / SPARQL** is dominant in Wikidata, biomedical knowledge graphs (UMLS, MeSH, ChEMBL), open government / FAIR-data initiatives, and any domain where the value proposition is *interchange*: well-defined vocabularies that travel between organizations and tools.
- **Property graphs / Cypher** is dominant in LLM-application backends (recipe stores, document KGs, agent memory), recommendation systems, fraud detection, and social-network analysis — domains where the value proposition is *engineering velocity*: schema you control, fast traversals, embeddable in Python.

Teaching both is what the program means by "graph fluency." A practicing engineer should be able to pick up either notation when they meet it.

2.1 The same 5-node fragment in both notations

A tiny recipe slice in RDF / Turtle:

```
:tomato a :Ingredient ; rdfs:label "tomato" .
:basil a :Ingredient ; rdfs:label "basil" .
:italian a :Cuisine ; rdfs:label "Italian" .
:caprese a :Recipe ; rdfs:label "Caprese" ;
    :usesIngredient :tomato , :basil ;
    :ofCuisine :italian .
```

The same five nodes and four edges as a Neo4j property graph:

```
MERGE (tomato:Ingredient:Entity {id: 'ingredient:tomato'}) SET tomato.name = 'tom
MERGE (basil:Ingredient:Entity {id: 'ingredient:basil'}) SET basil.name = 'basil'
MERGE (italian:Cuisine:Entity {id: 'cuisine:italian'}) SET italian.name = 'Italia
MERGE (caprese:Recipe:Entity {id: 'recipe:caprese'}) SET caprese.name = 'Caprese'
MERGE (caprese)-[:USES_INGREDIENT]->(tomato);
MERGE (caprese)-[:USES_INGREDIENT]->(basil);
MERGE (caprese)-[:OF_CUISINE]->(italian);
```

Same nodes, same edges, same query intent — two notations. The notation differences are real (URIs vs node ids, predicates vs typed relationships, triple shape vs ASCII-art) but they are syntactic; the underlying primitive is identical.

2.2 SPARQL ↔ Cypher mapping table

Concept	SPARQL (W9A)	Cypher (W9B)
Triple / single edge	`?s :usesIngredient ?o`	`MATCH (s)-[:USES_INGREDIENT]->(o)`
Identity	URI (`:tomato` , globally unique)	`(:Entity {id: 'ingredient:tomato'})` + uniqueness constraint (manual)
Type	`?x a :Recipe`	`(x:Recipe)` label
Hierarchical traversal	`?c rdfs:subClassOf* :Asian`	`MATCH (c)-[:SUBCLASS_OF*0..]->(:Cuisine {name: 'Asian'})`
Conditional filter	`FILTER (?prep < 30)`	`WHERE r.prepMinutes < 30`
Optional pattern	`OPTIONAL { ?r :requiresTechnique ?t }`	`OPTIONAL MATCH (r)-[:REQUIRES_TECHNIQUE]->(t)`
Sort + limit	`ORDER BY DESC(?p) LIMIT 10`	`ORDER BY r.popularityScore DESC LIMIT 10`

The rest of this reading uses Cypher. Read each query mentally as "the same shape I would write in SPARQL, with a different surface notation."

---

3. Cypher as pattern matching

A Cypher query is an **ASCII-art pattern** matched against the graph. Nodes are written as patterns inside parentheses, `(x:Label {property: value})`; relationships are written as arrows inside brackets, `-[:TYPE]->`. The engine finds every place in the graph where this pattern fits and returns what the query asks for.

3.1 Single-node lookup

Find every Recipe whose name is `"Caprese"`:

```
MATCH (r:Recipe {name: 'Caprese'})
RETURN r.name, r.popularityScore;
```

This is the property-graph analogue of SPARQL's `?r a :Recipe ; rdfs:label "Caprese"` triple pattern.

3.2 Single-hop traversal

Find every recipe that uses tomato:

```
MATCH (r:Recipe)-[:USES_INGREDIENT]->(:Ingredient {name: 'tomato'})
RETURN r.name;
```

Read this as the drawing `(Recipe) -[USES_INGREDIENT]-> (Ingredient with name 'tomato')`. Wherever the graph contains that shape, the recipe binds to `r`.

A **hop** is the traversal of a single relationship — exactly one `-[:TYPE]->` arrow in the Cypher pattern. The query above is a *single-hop* query: it crosses one `USES_INGREDIENT` edge to get from a recipe to an ingredient. A **multi-hop** query chains two or more relationship traversals back to back: *A* → *B* → *C* → …. "Who wrote this book?" is one hop. "Who are the co-authors of the people who cited this book?" is two hops: book → citers → co-authors of citers.

Multi-hop traversal is the structural query a property graph makes cheap, and it is the query a vector store does not have a native primitive for. M8's nearest-neighbor search returns passages whose embeddings sit close to the query embedding; nothing in that operation expresses "two more relationship steps." Chained vector retrievals can approximate it, but the graph treats the chained path as the primitive object — you ask for it directly, and the path itself is the deliverable, not just the endpoints.

3.3 Variable-length path traversal

Cypher's variable-length-path operator `*0..` is the property-graph mirror of SPARQL's `rdfs:subClassOf*`. The W9B cuisine taxonomy has 3–4 levels (`Sichuan → Chinese → Asian → World`); we want a query like "all Asian recipes" to cover Sichuan and Cantonese recipes too.

```
MATCH (r:Recipe)-[:OF_CUISINE]->(c:Cuisine)
MATCH (c)-[:SUBCLASS_OF*0..]->(:Cuisine {name: 'Asian'})
RETURN r.name;
```

The `[:SUBCLASS_OF*0..]` segment means *zero or more* `:SUBCLASS_OF` *hops.* Zero hops covers recipes whose direct cuisine is `Asian`; one hop covers `Chinese`; two hops covers `Sichuan`. The same pattern is essential for the Lab's entity linker: it lets a candidate of type `:Ingredient` match an NER (Named Entity Recognition) label of `FOOD` even when the NER label sits a few subclass levels above the Ingredient.

3.4 Parameterized Cypher — `$param`

When a query value comes from user input (a surface form, an NER span, a request parameter), **never** interpolate it into the Cypher string. Always pass it as a parameter:

```
# Correct — parameterized
query = "MATCH (e:Entity) WHERE toLower(e.name) = toLower($surface) RETURN e.id A
with driver.session() as session:
    rows = list(session.run(query, surface=user_input))
```

The string passed to `session.run` is constant; the value of `$surface` is supplied separately and bound by the driver. The driver escapes it correctly even if the surface form contains apostrophes, semicolons, or Cypher reserved words.

The buggy f-string counterpart looks innocent but is broken:

```
# Wrong — string interpolation, Cypher-injection vulnerability
query = f"MATCH (e:Entity) WHERE toLower(e.name) = toLower('{user_input}') RETURN
with driver.session() as session:
    rows = list(session.run(query))
```

A surface form like `O'Brien` produces malformed Cypher: the `'` closes the literal early and the parser fails. A maliciously chosen surface form like `'; MATCH (n) DETACH DELETE n; //` can do far worse on a writable connection. The fix is always the same: build a constant string with `$param` placeholders, pass values as keyword arguments to `session.run`.

This is the property-graph analogue of the SPARQL `initBindings` discipline from W9A. The Lab's autograder includes an AST (Abstract Syntax Tree) inspection test that fails if your `candidates()` function builds the Cypher string by f-string interpolation; passing the test means using `$param`.

---

4. Identity Discipline — the property-graph deliverable

In Week A, identity was free. Every node was a URI, and URIs are globally unique by construction — `:tomato` is `:tomato` no matter who minted it.

In a property graph, identity is **manual**. The database happily lets you create two distinct nodes that both have `name = 'orange'`. Without intervention, every ingest of "orange" appends a new node, your linker sees a growing list of candidates that all look the same, and the Lab's precision/recall numbers become noisy.

The mechanism that fixes this is a **uniqueness constraint** on a chosen identity property, paired with `MERGE` (idempotent create-or-match) instead of `CREATE`:

```
CREATE CONSTRAINT entity_id_unique FOR (n:Entity) REQUIRE n.id IS UNIQUE;
MERGE (e:Entity {id: 'orange:fruit'}) SET e.name = 'orange', e.type = 'Fruit';
```

Read line by line. The first line tells Neo4j: across all nodes carrying the `:Entity` label, the `id` property must be unique; the database will reject any attempt to create a second `:Entity` with the same `id`. The second line says: find a node with `id = 'orange:fruit'`, or create one if it doesn't exist; then set its `name` and `type`. Running the second line twice produces the same single node — `MERGE` is idempotent.

4.1 The `id` convention

Across Week B, every node carries the `:Entity` label *in addition to* its domain label. So a recipe is `(:Recipe:Entity)`, an ingredient is `(:Ingredient:Entity)`, and so on. The `:Entity` label is what the uniqueness constraint targets, so the discipline applies uniformly without having to declare a constraint per domain label.

The `id` convention is `'<label>:<slug>'`:

- `'recipe:42'` — a recipe with internal id 42
- `'ingredient:orange'` — the ingredient *orange*
- `'cuisine:sichuan'` — the cuisine *Sichuan*
- `'ingredient:szechuan_peppercorn'` — uses an underscore where a slug needs more than one word

The Lab provides a `canonical_id(label, name)` helper that builds this string. You use it; you don't re-implement it.

4.2 Why this matters

If you skip the constraint and run ingest twice, you end up with two `(:Ingredient {name: 'orange'})` nodes. The entity linker's candidate-generation step returns both of them; the disambiguator sees an "ambiguous" case where there should be only one candidate; the linker abstains; recall drops. Worse, the autograder's identity-uniqueness test fails — the Lab gates explicitly on it.

The Lab repo will ask you to mirror the snippet above as the foundation of its Identity Mapping utility — surface text → canonical `id`. The Reading is where you learn the discipline; the Lab is where you wire it in.

---

5. Entity linking on a property graph

This is the central algorithm in the Lab. Entity linking takes the output of NER (a span with a surface form like `"Sichuan"` and an NER label like `GPE`) and resolves it to a `(node_id, type_label)` in your knowledge graph — or to NIL if no plausible resolution exists.

The algorithm has four steps. Each step is a Cypher query (or a Python decision over Cypher results) and each step narrows the candidate set.

5.1 Candidate generation

For a surface form `s`, fetch every `:Entity` node whose `name` matches case-insensitively:

```
MATCH (e:Entity)
WHERE toLower(e.name) = toLower($surface)
RETURN e.id AS id, e.name AS name, labels(e) AS labels;
```

Pass `$surface` as a parameter (see §3.4). The result is a list of dicts; each dict's `labels` field is the node's label list with `:Entity` filtered out (`:Ingredient`, `:Cuisine`, etc.) — the type information the next step uses.

Real systems also implement fuzzy matching, altname tables, and SKOS preferred-label lookups; for W9B we keep candidate generation to exact case-insensitive matches plus the small altname table the Lab fixture seeds. The principle is the same.

5.2 Label-based type filter

If candidate generation returns multiple candidates, the second step narrows them by type compatibility with the NER label. spaCy's `en_core_web_sm` emits NER labels like `PERSON`, `GPE`, `ORG`, `FOOD` (via custom rule); your KG has its own label space (`:Recipe`, `:Cuisine`, `:Ingredient`, `:Author`, `:Technique`). The Lab maintains a small mapping:

```
NER_LABEL_TO_KG_TYPE = {
    "PERSON": {"Author"},
    "GPE":    {"Cuisine"},
    "FOOD":   {"Ingredient", "Cuisine"},   # ambiguous on purpose
    "ORG":    {"Author"},
}
```

For each candidate, keep it if any of its KG labels is in the allowed set. For `"Sichuan"` with NER label `GPE`, the allowed set is `{"Cuisine"}`; an `(:Ingredient {name: 'Sichuan'})` candidate (if it existed) would be dropped.

5.3 Hierarchical type compatibility — `[:SUBCLASS_OF*0..]`

The type filter alone is sometimes too narrow. The cuisine taxonomy has 4 levels:

```
World
├─ Asian
│  ├─ Chinese
│  │  ├─ Sichuan
│  │  ├─ Cantonese
│  │  └─ Hunan
│  ├─ Japanese
│  └─ Indian
├─ European
│  ├─ Italian
│  │  ├─ Tuscan
│  │  └─ Sicilian
│  └─ French
└─ Americas
   ├─ Mexican
   └─ NorthAmerican
```

Suppose the question is *"Find Asian recipes"*. The spaCy `GPE` for `"Asian"` resolves to `(:Cuisine {name: 'Asian'})`. The Lab's linker, when filtering candidates that themselves carry `:Cuisine` and live somewhere in this taxonomy, traverses `[:SUBCLASS_OF*0..]` to verify the candidate is in the Asian subtree:

```
MATCH (cand:Cuisine {id: $cand_id})
MATCH (cand)-[:SUBCLASS_OF*0..]->(:Cuisine {name: $ner_text})
RETURN cand.id AS resolved_id;
```

If the path exists (zero or more hops up the hierarchy), the candidate is hierarchically compatible. This is the W9B mirror of W9A's `rdfs:subClassOf*` reasoning beat — same primitive, Cypher syntax.

5.4 Property-neighborhood disambiguation

If two or more candidates survive type and hierarchy filtering, the linker disambiguates using the document's context — specifically, by counting how many of the document's already-linked entities co-occur in each candidate's 1-hop neighborhood:

```
MATCH (cand:Entity {id: $cand_id})-[*1..1]-(neighbor:Entity)
WHERE neighbor.id IN $resolved_ids
RETURN count(DISTINCT neighbor) AS overlap;
```

Read this as: "for this candidate, how many of its graph-neighbors are entities I've already resolved earlier in this document?" The candidate with the highest `overlap` wins. The Lab iterates over the document in span order so that `$resolved_ids` grows as you go — the co-occurrence signal builds up as the linker resolves more spans.

Tie-breaking: if the overlap is still tied, the Lab returns NIL with reason `nil-ambiguous`. We treat unresolved-by-context as a real abstention rather than a guess.

5.5 The recurring worked example

Throughout this section the Lab carries one running document: *"Yoshua Bengio gave a talk on deep learning at MILA in Montreal."* Candidate generation on `"Bengio"` returns one `(:Author)` candidate (resolved-unique). Candidate generation on `"Montreal"` returns three (`:City`, `:District`, `:Restaurant`); the NER label `GPE` plus a co-occurrence with the previously-resolved `(:Organization {name: 'MILA'})` picks the city. Candidate generation on `"deep learning"` returns zero — NIL.

---

6. The NIL case and linker metrics

Some surface forms have no plausible candidate in the KG. A brand-new product name, an unmodeled domain, a misspelling that doesn't match any altname — the right answer is **abstain**, not *guess wrong*. The linker emits NIL in these cases, and a NIL is a legitimate (and frequently correct) prediction.

The triple-stated methodology below is verbatim from the Lab's `linker/score.py` docstring, the Lab spec, and this reading — the same words appear in all three places. This is intentional: the Evaluation Methodology Rule requires that any function computing a metric state its methodology identically wherever it appears, so that a learner cannot accidentally implement a structurally-correct but semantically-wrong scoring function.

6.1 Triple-stated P/R/F1 methodology

- **Predictions are filtered to the gold span set (same `doc_id`, `start`, `end`) before scoring; predictions on spans absent from gold are dropped.**
- **A span is a true positive iff the predicted `(node_id, type_label)` EXACTLY matches gold AND gold is non-NIL.**
- **A prediction of a wrong `(node_id, type_label)` on a non-NIL gold is a false positive AND a false negative on that span.**
- **A NIL prediction on a non-NIL gold is a false negative only.**
- **A non-NIL prediction on a NIL gold is a false positive only.**
- **A NIL prediction on a NIL gold is a true negative (not counted in precision or recall).**
- **Aggregation is macro-average across documents (per-doc P/R/F1 averaged with equal weight per doc; docs with no gold spans are skipped).**

6.2 Reading the rules

A few subtleties are worth naming out loud. First, **NIL is asymmetric**: predicting NIL on a non-NIL gold costs you a recall point but not a precision point (you didn't claim anything wrong; you just failed to claim something right). Predicting a wrong identity on a non-NIL gold costs you both — once for the precision miss and once for the recall miss. Abstention is therefore strictly safer than guessing on the precision dimension.

Second, **the filtering step is what makes NIL on NIL count as a true negative without polluting the metric**. We score only on the gold span set, so predictions made on spans that gold never marked simply disappear before scoring. This is the standard CoNLL (Conference on Natural Language Learning) -style entity-linking convention.

Third, **macro-average means each document is equally important**. A 200-word document with 30 entities counts the same as a 30-word document with 3 entities. This is the right convention when documents come from different domains or have very different lengths — it stops a single long document from drowning the metric. The cost is that a document with 1 gold span and 1 prediction can register as P=R=F1=1.0 and pull the average up. That's why the Lab's dev split has 20 documents — enough samples that single-document quirks don't dominate.

---

7. Evaluation against gold

Gold annotations live in `data/dev.jsonl` (and the held-out `data/test.jsonl`) as records of the form:

```
{
  "doc_id": "doc_001",
  "spans": [
    {"start": 0,  "end": 8,  "surface": "Caprese",  "gold_node_id": "recipe:capre
    {"start": 14, "end": 19, "surface": "basil",    "gold_node_id": "ingredient:b
    {"start": 33, "end": 40, "surface": "umami",    "gold_node_id": null,
  ]
}
```

A `null` for both `gold_node_id` and `gold_type_label` means the gold annotation says NIL.

7.1 A toy worked example

Imagine a single document with 4 gold spans and your linker's 4 predictions:

span	gold	prediction	outcome
`"Caprese"`	`(recipe:caprese, Recipe)`	`(recipe:caprese, Recipe)`	TP
`"basil"`	`(ingredient:basil, Ingredient)`	`(ingredient:basil_plant, Ingredient)`	wrong identity → FP + FN
`"umami"`	NIL	NIL	TN-NIL (not in P or R)
`"Italian"`	`(cuisine:italian, Cuisine)`	NIL	NIL-on-non-NIL → FN only

Counting per the methodology: TP = 1, FP = 1, FN = 2.

Per-doc precision = TP / (TP + FP) = 1 / 2 = **0.500**. Per-doc recall = TP / (TP + FN) = 1 / 3 ≈ **0.333**. Per-doc F1 = 2·P·R / (P + R) = 2 · 0.5 · 0.333 / 0.833 ≈ **0.400**.

With one document, macro-average equals the per-document values. With many documents, each document's `(P, R, F1)` is computed independently and the three numbers are averaged with equal weight.

7.2 Macro vs micro

Macro-average (the Lab's choice) treats every document as a single vote. Micro-average pools the per-document counts (sum TP across docs, divide by total predicted; sum TP, divide by total gold) and is dominated by long documents with many spans. Use macro when document lengths vary and you want each document to contribute equally; use micro when you want a single per-span weighting.

The Lab uses macro to match how the gold corpus was constructed — each document is its own annotation unit.

---

8. Text2Cypher — schema-bounded NL→code generation

Text2Cypher is the canonical pattern for letting a user (or an LLM agent) ask a question in natural language and receive an answer backed by a knowledge graph. The flow is: NL question → bounded schema context → emitted Cypher → executed against Neo4j → returned rows → composed answer.

8.1 The transferable primitive

Text2Cypher is one instance of a broader transferable engineering pattern: **schema-bounded NL→code generation**. A system that takes natural language as input and emits structured code or queries against a known, bounded schema. The same primitive shows up under several names:

- **NL→SQL (Structured Query Language)** — data-analytics agents, BI (Business Intelligence) copilots, in-product analytics
- **NL→API (Application Programming Interface) -call** — LLM tool use / function calling; "format-locked" structured outputs
- **NL→Cypher** — this week
- More generally: any LLM call that emits JSON conforming to a schema, function-call arguments, regex, or any other constrained surface

The unifying engineering question is the same in every instance: *how do I get a reliable, schema-conformant string out of an NL input?* The schema-boundedness is what makes the question tractable — the smaller and more known the target language is, the more options you have.

8.2 Two production-grade implementations

Both of the following are first-class implementations of the schema-bounded NL→code primitive on the W9B Cypher surface. They differ in *where the generation happens*, not in *whether they are production patterns*.

Live-LLM Text2Cypher — LangChain `GraphCypherQAChain`. A schema preamble (the labels, the relationship types, and 2–3 example MATCH patterns) plus a few-shot block (NL → Cypher pairs that illustrate the patterns you want) plus the user's NL question is sent to an LLM; the LLM emits a Cypher string; the system executes the string against Neo4j via Bolt; the rows are composed into an NL answer. This is the pattern in production codebases that ship "ask the database in English" features against schemas that change frequently or are too broad to template.

```
from langchain_neo4j import Neo4jGraph, GraphCypherQAChain
from langchain_community.chat_models import ChatOllama

graph = Neo4jGraph(url="bolt://localhost:7687", username="neo4j", password="testt
llm = ChatOllama(model="phi3:mini-4k-instruct-q4_K_M")
chain = GraphCypherQAChain.from_llm(llm=llm, graph=graph, schema=SCHEMA_PREAMBLE)
chain.invoke({"query": "Find Sichuan recipes that use ginger"})
```

Deterministic NL→Cypher mapping. A rule-based intent classifier + slot extractor + Cypher template compiler. The mapper detects which of N supported question shapes the NL matches (e.g., "single-cuisine recipes"), extracts the slot values from the NL (e.g., `cuisine = 'Sichuan'`), and compiles the result into a parameterized Cypher template. No LLM call is made at runtime; the same input always produces the same output. This is the pattern in production codebases where the schema is bounded enough to template, the input distribution is known, and grading-stable / deployable behavior is a hard requirement.

```
from mapper.intent import detect_shape
from mapper.slots import extract_slots
from mapper.compile import compile_to_cypher

shape = detect_shape("Find Sichuan recipes that use ginger")
slots = extract_slots("Find Sichuan recipes that use ginger", shape)
cypher, params = compile_to_cypher(shape, slots)
# cypher is a constant template with $-parameters; params is the slot dict
```

Both implementations ship the same NL→code primitive. The choice between them is an engineering judgment call about your input distribution: is the schema bounded enough to template? is the NL distribution narrow enough to predict? are grading-stability or deployment-determinism required? Both answers are legitimate; neither is the "real" implementation and the other a placeholder.

8.3 A worked failing example — measure before committing

To make the design judgment concrete, the following is the multi-hop conjunction question *"Find Sichuan recipes that use ginger and require wok technique"* with two implementations side by side.

Plausibly-failing small-LLM Cypher (reversed subclass direction or missing conjunction):

```
MATCH (r:Recipe)-[:OF_CUISINE]->(c:Cuisine {name: 'Sichuan'})
MATCH (c)<-[:SUBCLASS_OF*0..]-(:Cuisine)
MATCH (r)-[:USES_INGREDIENT]->(:Ingredient {name: 'ginger'})
RETURN r.name;
```

This parses, runs without error, and returns rows. But the second `MATCH` reverses the subclass arrow direction (`<-[:SUBCLASS_OF*0..]-` instead of `-[:SUBCLASS_OF*0..]->`), so it traverses *down* the cuisine tree from Sichuan rather than *up* — which means it finds Sichuan-or-subtype recipes, but it loses the conjunction's third clause entirely: `wok technique` is never restricted. The result set looks plausible but contains recipes that do not require wok technique.

Deterministic mapper output (correct):

```
MATCH (r:Recipe)-[:OF_CUISINE]->(c:Cuisine)
MATCH (c)-[:SUBCLASS_OF*0..]->(:Cuisine {name: $cuisine})
MATCH (r)-[:USES_INGREDIENT]->(:Ingredient {name: $ingredient})
MATCH (r)-[:REQUIRES_TECHNIQUE]->(:Technique {name: $technique})
RETURN r.name;
```

```
params = {"cuisine": "Sichuan", "ingredient": "ginger", "technique": "wok"}
```

All three constraints are present, the subclass direction is right, and the parameter binding keeps the surface forms separate from the query string. The deterministic mapper is correct by construction because the template is checked in once and tested against the gold answer set.

8.4 The engineering principle

The contrast above is not a claim about Phi-3 specifically. It illustrates a deeper engineering principle: *measure before committing*. The W9B Integration's design includes a documented verification gate — a 15-question evaluation of two candidate small LLMs (Phi-3 Mini at 60.0% valid-Cypher emission; Qwen 2.5 1.5B at 66.7%; the project's deployment threshold ≥80%). Both fell short of the threshold, so the Integration's core path is the deterministic mapper, with the live-LLM variant available to learners who want to study it as Tier 3. The same measurement on a different schema, a larger LLM, or a narrower question distribution would land differently, and a different routing decision would follow.

The discipline is: before you commit to an implementation, characterize the input distribution and measure the candidates against it. Then build to what the measurements support.

8.5 White-box vs black-box retrieval

A second reason to take the deterministic mapper and the Cypher query itself seriously — not just the LLM — is **explainability**. Both implementations of Text2Cypher above end in a Cypher string and a row set. The Cypher string is a **white box** — a system you can see inside of, where every step is visible. A learner, a TA, or a downstream service can read it, point at each clause, and trace *why* a particular row came back — *this `MATCH` bound the recipe, this `WHERE` filtered to the cuisine, this `[:SUBCLASS_OF*0..]` reached Sichuan from Asian.* The answer carries its provenance.

Contrast this with the **black box** of M8's vector retrieval — a system where you see the input and the output but not the steps between. When a semantic search returns a passage, the system can report a similarity score, not a traceable path: it cannot point at the specific feature dimensions that made the embedding close. For ranking the next thing to read, that is enough. For grounding a claim a downstream system will act on — a medical recommendation, a compliance decision, an answer cited inside another pipeline — the white-box path is what lets you justify the answer.

One caveat. White-box traceability is about *provenance*, not *truth*. If the data in the graph is wrong — a mislabelled ingredient, a missing relationship, an outdated author attribution — the white-box query confidently returns the wrong answer along a perfectly auditable path. Data quality is a separate, upstream problem that no query language solves.

This is the engineering pay-off of pairing M8 and M9. Vectors find candidates fast. Graphs trace the path. A production pipeline that uses both — vector retrieval to widen the recall surface, graph traversal to verify and explain the path — gets the best of both substrates.

---

9. Text2Cypher vs full GraphRAG

Two retrieval patterns share the property that they involve a graph, but they answer different kinds of questions.

Text2Cypher (W9B core integration). NL → Cypher → graph → answer. A structured query against a structured store. Best when the question can be expressed as a pattern against the schema — entity lookups, conjunctions, aggregations, hierarchy traversals. The whole point is the precision of structured query.

Full GraphRAG (W9B Stretch Thu, Honors). NL → vector retrieval *and* Cypher traversal, fused at answer time. Vector retrieval finds candidate nodes by semantic similarity to the question; graph traversal expands each candidate's local neighborhood; a fusion step ranks the combined set. Best when the question is partly semantic ("recipes like this one") and partly structural ("…that also use a Sichuan-style technique") — that is, when neither pure vector search nor pure Cypher answers it cleanly.

Both rest on Neo4j 5.x: the same engine ships a native vector index since 5.11, so a hybrid system is one schema and one driver. The M8 vector primitive carries forward; only the engine differs (Weaviate → Neo4j vector index).

---

10. Text2Cypher safety (Tier 3 only)

When a learner enables Tier 3, the live LLM may emit any Cypher string that parses. Most of what it emits will be a read query, but the LLM is a probabilistic system and adversarial inputs (or even an unlucky prompt) can produce destructive Cypher. Tier 3 ships two layers of defense.

10.1 Primary defense — query-shape allowlist

A small Python module inspects every LLM-emitted Cypher string before it reaches the driver. Allowed clauses are read-only: `MATCH`, `OPTIONAL MATCH`, `WHERE`, `RETURN`, `WITH`, `ORDER BY`, `LIMIT`, `UNION`. Forbidden clauses are write or admin: `CREATE`, `MERGE`, `DELETE`, `DETACH DELETE`, `SET`, `REMOVE`, `DROP`, and any procedural `CALL` (which can invoke APOC-style export or admin procedures). A Cypher string containing any forbidden clause is rejected before it reaches Neo4j, raising `UnsupportedCypherError` with the offending clause named.

10.2 Defense-in-depth — read-only connection

The Tier 3 docker-compose creates a `reader` user (alongside `neo4j` / `testtest` for fixture loading) and the LLM-facing chain connects as `reader`. Neo4j Community Edition does not support fine-grained per-user role-based access control — `reader` cannot have a Neo4j-side restriction; the Python allowlist is the primary safety surface. The reader-vs-admin separation is defense-in-depth, not the primary surface.

10.3 Walked adversarial prompt

A user types: *"delete all recipes."*

The LLM is well-trained enough to often emit a refusal, but not reliably. A small instruct model under a few-shot Cypher-emission prompt may emit:

```
MATCH (r:Recipe) DETACH DELETE r;
```

The allowlist module tokenizes the Cypher (stripping quoted strings and comments first to avoid flagging the word `DELETE` inside a quoted name), uppercases each keyword, and checks against `FORBIDDEN_CLAUSES`. It hits `DETACH` and `DELETE` and raises:

```
UnsupportedCypherError: Forbidden clause 'DELETE' in LLM-generated Cypher;
only read clauses permitted.
```

The Cypher never reaches the driver; the database is unmodified. The autograder reports this case as "rejected by allowlist" separately from "executed but returned wrong rows" so a learner can distinguish unsafe emissions from safe-but-wrong ones.

---

11. KG completion as a learning problem

Every knowledge graph is incomplete. There are entities you have not modeled, relationships you have not extracted, edges you have not added yet. **KG completion** is the task of predicting the missing pieces.

11.1 CWA vs OWA

Two stances on missing edges:

- **Closed-World Assumption (CWA).** Absence of an edge means absence of the relationship. If `(:Recipe {name: 'Caprese'})` has no `-[:USES_INGREDIENT]->` edge to `(:Ingredient {name: 'olive oil'})`, then Caprese does not use olive oil. Simple, easy to query, *brittle*: every gap in your data is treated as a negative fact.
- **Open-World Assumption (OWA).** Absence of an edge means *unknown*. Caprese might or might not use olive oil; we just haven't recorded one way or the other. More honest about the data; harder to query naively because "not found" no longer means "false."

The choice is not philosophical — it has direct downstream consequences. A retrieval pipeline that assumes CWA on an OWA-natural knowledge graph will confidently return wrong answers ("we have no recipes that use olive oil"). A completion model softens the gap by predicting which absent edges are likely-true.

11.2 Link prediction

The canonical task framing for KG completion is **link prediction**: given a query like `(head, relation, ?)`, predict the tail entity. For W9B's recipe graph, link prediction would tackle questions like *"What ingredient is this recipe most likely missing?"* The answer is not extracted from text; it's predicted from graph structure.

A trained KG-completion model can also score *plausibility* of an existing edge, which is useful for cleaning up extracted relations that may be noisy. The model itself is typically a graph neural network — the next section.

---

12. GNN literacy — GAT and R-GCN

GNNs (graph neural networks) compute node embeddings by **message passing**: every node updates its representation by aggregating messages from its neighbors. After several rounds of message passing, a node's embedding captures the structure of its local neighborhood — which neighbors it has, which kinds of edges connect them, what those neighbors look like.

Two architectures show up repeatedly in KG contexts.

12.1 GAT — Graph Attention Network

GAT introduces **attention** to the aggregation step. Instead of treating every neighbor equally, the node learns a soft weight `α_{ij}` that says how much neighbor `j` matters for node `i` 's update. The per-neighbor weight is computed from a learnable attention function over the pair `(h_i, h_j)` and normalized across the neighbor set.

The attention coefficient (one equation):

```
α_{ij} = softmax_j(  LeakyReLU( a^T · [W h_i || W h_j] )  )
```

where `h_i`, `h_j` are node features, `W` is a learnable transformation, `a` is a learnable attention vector, `||` is concatenation. The softmax normalizes over the neighbor set so the weights for one node's neighbors sum to 1.

In prose: each neighbor's contribution to node `i` 's new embedding is weighted by how much node `i` "attends to" that neighbor. After training, the weights pick up structural patterns — neighbors of a more informative type get higher weight than uninformative neighbors. The output is a node embedding that summarises which of `i` 's neighbors matter for the downstream task.

12.2 R-GCN — Relational Graph Convolutional Network

R-GCN handles **typed edges** explicitly. Every edge type (`:USES_INGREDIENT`, `:OF_CUISINE`, `:SUBCLASS_OF`, …) gets its own learnable weight matrix `W_r`; messages along edges of type `r` are transformed by `W_r` before aggregation.

The per-relation aggregation (one equation):

```
h_i' = σ(  Σ_{r in R} Σ_{j in N_i^r}  (1 / |N_i^r|) · W_r · h_j  +  W_0 · h_i  )
```

where `R` is the relation set, `N_i^r` is the neighbors of `i` connected by relation `r`, `W_r` is the relation-specific transformation, `W_0` is a self-loop transformation, `σ` is a nonlinearity.

In prose: a node aggregates separately from each kind of neighbor, mixing the per-relation contributions before applying a nonlinearity. The output embedding is sensitive to *what kinds of relationships* the node sits in — exactly the distinction a property graph encodes. R-GCN is the natural neural architecture when relation types carry semantic load.

12.3 What these embeddings get used for

Downstream tasks for GAT / R-GCN embeddings include:

- **Link prediction** — score a hypothesised edge `(h, r, t)` by combining the embeddings of `h` and `t` under relation `r`.
- **Entity classification** — predict a node's label from its embedding.
- **Candidate ranking in entity linking** — when the W9B linker has two candidates that pass type + hierarchy filters, a GNN-trained ranking score over the candidates' embeddings can break the tie. The W9B Lab does not use a GNN — the ranking signal is property-neighborhood overlap — but the embedding-as-ranker pattern is what GNNs slot into.

There is no PyTorch code in this section. GNNs are a literacy topic for Week B; the equations above are here so that when you meet GAT or R-GCN again post-program — in a paper, a vendor pitch, or a notebook from a peer — the math is not foreign.

---

13. Neural–symbolic integration

The W9B integration assembles a small neural-symbolic system. The Reading frames three composition patterns; the Integration implements the first.

Pattern 1 — neural outputs grounded in symbolic structure. A neural component (spaCy NER) emits structured outputs (spans with labels); a symbolic component (the Cypher entity linker) resolves each span to a `(node_id, type_label)` in the KG; a second symbolic component (the deterministic NL→Cypher mapper) routes the user's question against the schema; the symbolic store (Neo4j) executes the query. The neural piece extracts; the symbolic piece grounds and reasons. **This is the W9B core integration.**

Pattern 2 — symbolic structure enriched by neural embeddings. A symbolic component (a KG) is augmented by neural embeddings (node embeddings from a GNN, or text embeddings from a sentence transformer) so that ranking, retrieval, or completion can be done in vector space while still respecting graph structure. The W9B Stretch Thu's GraphRAG hybrid is a taste of this pattern.

Pattern 3 — end-to-end systems that alternate. Retrieve, reason, refine — a system that loops between neural retrieval, symbolic reasoning, and neural composition. Agentic systems with tool use sit here; this is the active frontier you will encounter post-program.

The W9B Integration is intentionally Pattern 1 because grounding NER output in a KG is the canonical entry point — every more sophisticated pattern depends on it. Once Pattern 1 is solid, Patterns 2 and 3 become composable extensions rather than from-scratch builds.

---

14. The Week B Python toolkit

Core (everyone):

- **`neo4j`** — the official Bolt driver. Opens sessions, runs Cypher with `$param` bindings, returns rows as dicts.
- **`spacy`** — NER from M6; `en_core_web_sm` is the model. Used to extract `(start, end, surface, label)` records from text before linking.
- **`pytest`** — the autograder framework, as in every module.
- **`python-dotenv`** — read `NEO4J_URI` / `NEO4J_USER` / `NEO4J_PASSWORD` from `.env` in local dev.
- **`argparse`** (stdlib) — the CLI front-end for the Integration's `cli.py`.

Challenge Tier 3 only (opt-in):

- **`langchain`** — the orchestration framework for the live-LLM chain. Provides `GraphCypherQAChain`.
- **`langchain-neo4j`** — the Neo4j integration package. Provides `Neo4jGraph` (schema introspection + Cypher execution from a chain).
- **`langchain-openai`** / **`langchain-anthropic`** — provider integrations for the hosted-API paths (set `OPENAI_API_KEY` or `ANTHROPIC_API_KEY`). The Integration's `challenge/llm_client.py` resolves to whichever path your environment supplies.
- **`ollama`** — the Python client for the local Ollama runtime. Used by `langchain_community.chat_models.ChatOllama` (imported from `langchain-community`, which arrives transitively with `langchain`).

You install the core deps for every Week B assignment. You install the Tier 3 deps once, when you decide to attempt Tier 3 of the Integration. Tier 3 has no autograder dependency on the live LLM in CI — the autograder uses a cached mock response file — so a learner who installs Tier 3 locally can still run the same autograder the cohort sees.

Troubleshooting

Neo4j Bolt connection refused. Either the container is not running (`docker ps` ; if absent, `docker compose up -d`), or the container is running but the JVM (Java Virtual Machine) has not finished warming up. The Lab's autograder has a readiness-probe step (60 retries × 2s) that handles the latter; in local dev, wait 20 seconds after `docker compose up -d` before running tests, or watch `docker compose logs -f neo4j` for the line `Started.`

JVM warm-up race — first test run fails, second passes. Same root cause as above. Add a readiness probe to your local script if you keep hitting this, or simply re-run pytest once the container has warmed up.

Cypher injection symptoms — surface form with apostrophe crashes the parser. You are interpolating the surface form into the Cypher string. Switch to `$param` binding (see §3.4). The autograder's AST check catches f-string Cypher.

Duplicate `(:Entity)` nodes silently inflating recall. You loaded the fixture twice without the uniqueness constraint, or you used `CREATE` instead of `MERGE`. Apply the constraint from §4 first, then re-load with `MERGE`. The Lab's `test_entity_id_uniqueness` test catches this.

Linker abstains on everything — precision = 1.0, recall = 0.0. Your `disambiguate()` is returning `(None, "nil-ambiguous")` for every input. Likely cause: the type filter set is empty or the hierarchical traversal is querying the wrong direction. Check `NER_LABEL_TO_KG_TYPE` and the arrow direction in your `[:SUBCLASS_OF*0..]` Cypher.

`UnsupportedQueryError` from the deterministic mapper. The NL question does not match any of the 15 supported shapes — by design. The mapper raises a fail-loud error naming the supported shapes; adding a 16th template is the engineering exercise. This is not a bug.

Tier 3 only — Ollama model not pulled. `get_llm_client()` probes the local Ollama runtime and raises `OllamaModelMissingError` with the exact pull command. Run the suggested `ollama pull` command and retry.

Key Terms

property graph, node label, relationship type, Cypher, `MATCH`, ASCII-art pattern, variable-length path, `[:SUBCLASS_OF*0..]`, `$param` parameterization, Bolt driver, Neo4j Browser, Identity Discipline, uniqueness constraint, `MERGE`, entity linking, surface form, candidate generation, disambiguation, NIL, gold annotation, precision, recall, F1, macro-average, micro-average, closed-world assumption (CWA), open-world assumption (OWA), KG completion, link prediction, GNN, message passing, GAT, attention coefficient, R-GCN, relation-specific transform, node embedding, neural-symbolic integration, Text2Cypher, schema-bounded NL→code generation, schema preamble, few-shot prompt, `GraphCypherQAChain`, Ollama, query-shape allowlist, `UnsupportedQueryError`, `UnsupportedCypherError`, `LLM_PROFILE`.

Summary

Week B is the same primitive as Week A — graph as pattern matching — in a second production stack. The notation is Cypher; the engine is Neo4j; the schema is yours to control. Identity, which was free in RDF, becomes a deliberate piece of engineering: a uniqueness constraint plus `MERGE` over a stable `id` convention. Entity linking on the property graph is a four-step Cypher pipeline (candidates, label filter, hierarchical compatibility, neighborhood disambiguation) with explicit NIL handling and a triple-stated P/R/F1 methodology. Text2Cypher is the canonical NL→code pattern, and the choice between a live-LLM implementation and a deterministic mapper is an engineering judgment about your schema and input distribution — both are production-grade. KG completion frames the closed-world / open-world tension; GNN literacy gives you the GAT and R-GCN equations to recognize. The Week B integration composes a neural-symbolic system: spaCy NER → Cypher linker → deterministic mapper → Neo4j.

What's Next

Module 10 is **deployment**: FastAPI services, container orchestration, the production shape of the systems you have spent Weeks 8 and 9 building. Notice that the W9B Integration's deterministic NL→Cypher mapper is already a natural FastAPI service body — an input string, a structured output (Cypher template + parameter dict), a known set of supported shapes, fail-loud behavior on out-of-scope inputs. The Tier 3 live-LLM chain is the LLM-backed variant of the same endpoint. M10 turns these into deployable HTTP services with explicit request/response schemas, health checks, and the rest of the production surface.

The Week A reading (the RDF / SPARQL half) lives at the Week A reading page. Re-read §1–§3 of W9A next to §2 of this reading if the SPARQL ↔ Cypher correspondence has not yet settled.

© 2026 LevelUp Economy. All rights reserved. Unauthorized reproduction or distribution of this material is prohibited.